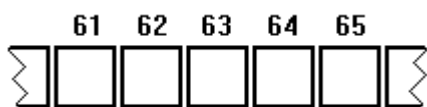


CHAPITRE IV – ALLOCATION DYNAMIQUE DE LA MEMOIRE

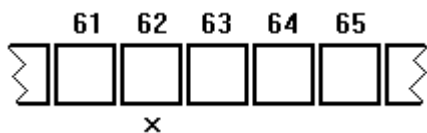
1. Présentation du concept

Les variables sont stockées dans une mémoire statique : on peut **représenter cette mémoire par des cases numérotées dans l'ordre croissant**. Le numéro de la case est appelé « adresse ». Dans ce qui suit, et pour faciliter les choses, on considérera que toute variable utilise une seule case, même si ce n'est pas vraiment conforme à la réalité. En effet, la quantité de mémoire prise par une variable dépend de son type. Il est utile de rappeler, dans ce contexte, que dans une machine **8 bits**, les adresses sont codées sur un octet, ce qui nous fait **256 cases numérotées de 0 à 255**.

Soit le schéma suivant :

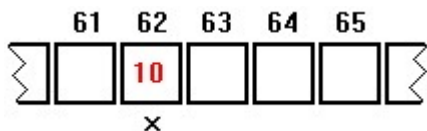


Quand on écrit : **x : entier**



On réserve une case pour la variable x dans la mémoire, case numéro 62 dans le cas du schéma.

Quand on écrit : **x <- 10**



On écrit la valeur 10 dans l'emplacement réservé pour x. Dans le schéma ci-dessus, cette valeur est située à la case 62.

Quand on exécute un programme P, le système d'exploitation se charge d'allouer l'espace mémoire nécessaire pour le stockage de toutes les variables déclarées dans P. Dans le cas où une variable déclarée ne sera pas utilisée ou qu'elle ne sera pas utile pendant toute la période d'exécution du programme, il y a un **gaspillage** de la mémoire. En effet, il y a deux types d'allocation mémoire : allocation statique (gérée automatiquement par le système) et dynamique (gérée manuellement par le programmeur).

L'allocation dynamique nous permet d'éviter ce type de gaspillage de mémoire et aussi de gérer les ensembles dynamiques (on verra cela dans le cours listes chaînées) et ce grâce aux pointeurs.

2. Les pointeurs

Un pointeur est une variable destinée à contenir une adresse mémoire, c'est à dire une valeur identifiant un emplacement en mémoire.

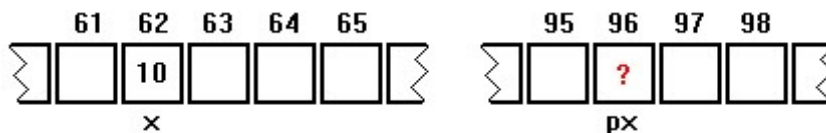
La déclaration d'une variable pointeur réserve uniquement le nombre d'octets nécessaires au codage de l'adresse mémoire de la donnée pointée mais ne réserve pas de la mémoire pour celle-ci (la donnée pointée). Quel que soit le type de la donnée pointée, la taille mémoire du pointeur est toujours la même.

a. Déclaration d'un pointeur

id_var_pointeur : ^ type de base

Exemple :

px : ^ entier



On réserve un emplacement en mémoire pour le pointeur px (case numéro 96 dans le cas du schéma).

b. Réservation de la mémoire

allouer (id_var_pointeur)

Exemple :

px : ^entier

allouer (px) $\Leftrightarrow$ z : entier et px <- adresse(z) // px pointe vers z et z est la variable pointée par px.

A la création deux choses se produisent :

- * De la mémoire est réservée pour stocker une donnée du type spécifié (entier pour px).
- * La variable concernée (px) contient l'adresse mémoire de cet espace réservé (représenté par une flèche).
- * px contient l'adresse de l'espace réservé, nous pourrons alors utiliser cette variable.

c. Accès à la variable pointée

On accède à la valeur pointée comme suit :

id_var_pointeur ^

Exemple :

px : ^entier

px^ : variable de type entier

px^ <- 20

d. Libération de la mémoire

Dès que le ou les pointeurs ne sont plus utiles, l'espace mémoire qui leur est associé doit être libéré. La libération de l'espace mémoire réservé est la procédure "réciproque" de **allouer**. Elle indique au système d'exploitation que cette partie de la mémoire dynamique n'est plus utilisée (*il peut désormais en faire ce qu'il veut*).

Exemple :

Libérer (px) // la zone pointée par px est libre de nouveau

Remarques

- * Pour indiquer l'absence d'adresse dans un pointeur, on utilise le pointeur particulier Nil.
- * L'affectation p <- Nil veut dire que p ne pointe vers aucune variable.
- * Il ne faut jamais utiliser l'indirection (^) lorsque le pointeur n'a pas été créé, a été déjà libéré, ou contient Nil.

Exp**Var**

p : ^entier

Debut

/* le contenu de p est indéterminé : une adresse quelconque de la mémoire

P^ <- 10 /* Erreur

Fin

- * Il est tout de même interdit de référencer une variable dynamique après sa destruction.

Var

p : ^entier

Debut

allouer(p)

P^ <- 10

Libérer(p)

P^ <- 20 /* Erreur : affectation d'un entier (20) à une zone indéterminée

FI

CHAPITRE V – LES FICHIERS

Introduction

Toutes les structures de données que nous avons utilisées jusqu'à maintenant étaient stockées en mémoire, c'est-à-dire dans une zone volatile de la machine. En mémoire, la durée de vie d'une variable est égale au temps d'exécution du programme. Nous comprenons facilement que cette solution n'est pas envisageable pour l'informatisation d'une entreprise par exemple. Les données concernant un employé, un produit, un fournisseur ou un contrat doivent être mémorisées plus longtemps que la durée du programme. Les valeurs traitées doivent être recopiées sur un support non volatile (disque dur, CD ROM...).

Mais quelle structure utiliser pour stocker ce type de données ?

Les fichiers répondent bien à ce besoin. Ils servent à stocker des informations de manière permanente, entre deux exécutions d'un programme.

1. Notion de fichier

Un FICHIER (anglais : file) est un REGROUPEMENT LOGIQUE DE DONNEES MEMORISEES SUR UN SUPPORT PERMANENT (disque dur, par exemple) afin de permettre une réutilisation ultérieure des informations qu'il contient.

2. Organisation des fichiers

a. Fichiers structurés (fichiers texte)

Les fichiers structurés sont composés d'un ensemble de données élémentaires identifiables de manière précise au sein du fichier, et qui sont répétés pour former ce fichier. On parle alors d'**enregistrement** ou article ou enregistrement logique.

Ce type de fichier est couramment utilisé dès lors que l'on doit stocker des informations pouvant être assimilées à une base de données. Par exemple, si le fichier est destiné à mémoriser les coordonnées d'un certain nombre de personnes. Pour chacune, il faudra noter le nom, le prénom, le numéro de téléphone et l'e-mail. Dans ce cas, il peut paraître plus simple de stocker une personne par ligne du fichier (par enregistrement). Dit autrement, quand on prendra une ligne, on sera sûr qu'elle contient les informations concernant une personne, et uniquement cela. Cette catégorie est aussi appelée fichier texte. Un fichier texte est constitué par une collection d'enregistrements : répétition de la structure ligne.

b. Fichiers non structurés (fichiers binaires)

Le second type de fichier rassemble les fichiers qui ne possèdent pas de structure d'enregistrement. Les octets, quels qu'ils soient, sont écrits à la queue leu leu. Ces fichiers sont appelés des fichiers binaires.

➔ Exemple : Tous les fichiers qui ne codent pas une base de données sont obligatoirement des fichiers binaires tels que : un fichier son, une image, un programme exécutable...

3. Accès aux fichiers

Un autre critère de distinction entre les fichiers consiste en le type d'accès. On distingue principalement l'accès séquentiels et l'accès direct ou aléatoire.

Accès séquentiel : Un fichier séquentiel permet uniquement d'accéder aux données dans l'ordre de leur écriture : On ne peut accéder qu'à la donnée suivant celle qu'on vient de lire. On ne peut donc accéder à une information qu'en ayant au préalable examiné celle qui la précède. Dans le cas d'un fichier texte, cela signifie qu'on lit le fichier ligne par ligne (enregistrement par enregistrement).

Accès direct (ou aléatoire) : on peut accéder directement à l'enregistrement de son choix, en précisant le numéro de cet enregistrement. Mais cela veut souvent dire une gestion fastidieuse des déplacements dans le fichier. Un fichier à accès direct correspond à un tableau en mémoire : toutes ses composantes ont la même taille, on peut donc accéder directement à la Nième. Comme pour un tableau, les insertions et suppressions nécessitent des décalages des composantes suivantes, ce qui est en général trop long pour être envisageable.

Dans ce qui suit, nous nous intéresserons aux fichiers séquentiels.

4. Déclaration**Définition de type**

Fil = fichier de reel (le type des éléments du fichier est reel)

Déclaration de variables

F : Fil ou F : fichier de reel

X1	X2	X3	...	Xn	EOF ou FDF
----	----	----	-----	----	------------

X1...Xn sont des variables ayant le même type (ou un type compatible) avec le type figurant dans la déclaration du fichier.

5. Instructions**a. Ouverture d'un fichier**

Si l'on veut travailler sur un fichier, la première chose à faire c'est de l'**ouvrir**. L'instruction d'ouverture doit indiquer le mode d'ouverture du fichier. La forme générale de l'instruction est la suivante : **Ouvrir (variable fichier, mode d'ouverture)**

- Si on ouvre un fichier en mode **lecture**, on pourra uniquement récupérer les informations qu'il contient, sans les modifier.

Exp : ouvrir (F, lecture)

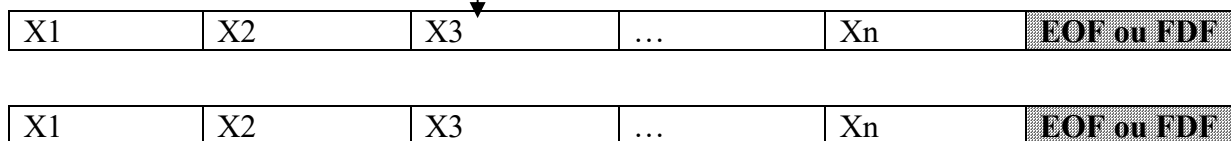
EOF (End Of File) ou FDF (Fin De Fichier) est une variable booléenne :

- = faux s'il existe encore des enregistrements et
- = vrai si la fin du fichier est atteinte.
- Si on ouvre un fichier en mode **écriture**, on pourra y écrire des informations ; mais les informations précédentes, si elles existent, seront **intégralement écrasées**.
- Si on ouvre un fichier en mode **ajout**, on pourra y ajouter des enregistrements sans écraser les informations déjà existantes. Par contre, ces informations ne pourront pas être lues, ni modifiées.

b. Lecture depuis un fichier

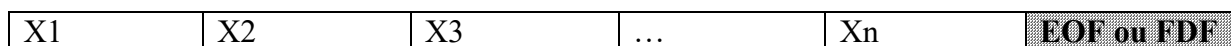
L'instruction de lecture d'un fichier récupère donc dans la variable spécifiée l'enregistrement suivant dans le fichier. « Suivant » par rapport au dernier enregistrement lu. C'est pour cela que le fichier est dit séquentiel.

Tête de lecture est actuellement dans cette position par exemple, donc on va lire X4



Lire (F,x) $\Leftrightarrow$ x $\leftarrow$ F

Exemple :



Ouvrir (F, Lecture)

Lire(F,x)

Tant que non FDF(F) faire

Ecrire(x)

Lire (F,x)

FTQ

➔ Va afficher X1, X2, X3, etc

c. Ecriture dans un fichier

Ouvrir (F, Ecriture)

Ecrire (F,x) $\Leftrightarrow$ x est écrite dans le fichier

Exemple:

Ecrire un algorithme qui permet de saisir une suite de 10 éléments et de les écrire dans un fichier.

Algorithme Ecriture ()**Var**

x,i : entier

F : Fichier

Debut

Ouvrir (F, Ecriture)

Pour i de 1 à 10 Faire

 Lire(x)

 Ecrire (F,x)

Fpour

Fin**d. Fermeture d'un fichier**

La dernière instruction du programme est la fermeture du fichier : Fermer(F)

6. Exercices

- a. Ecrire une procédure permettant de créer et remplir le fichier (de type Fi_employés) qui contient des informations sur les employés d'une entreprise (matricule, nom, prénom, grade, salaire).
- b. Ecrire une procédure permettant d'afficher la liste des employés à partir du fichier de type Fi_employés.
- c. Ecrire une fonction permettant d'afficher la liste des employés dont le salaire est compris entre 500 et 700 DT.
- d. Copier les salaires des employés dans un tableau T_Salaire.

CHAPITRE VI - LES TYPES ABSTRAITS DE DONNEES

1. Présentation du concept de TAD

La conception d'un algorithme de complexité moyenne ou élevée se fait en plusieurs étapes pendant lesquelles on effectue des raffinements successifs de la solution proposée.

La première version d'un tel algorithme est en général indépendante de son implémentation. En particulier, la représentation des données n'est pas fixée. A ce premier niveau, les données sont donc décrites par une notation particulière, un ensemble d'opérations ainsi que leurs propriétés et sont manipulées de façon abstraite c'est-à-dire non liée à la représentation. C'est exactement ça le concept de **Type Abstrait de Données** que nous désignerons dans la suite par TAD.

Dans les étapes ultérieures, les différentes représentations du TAD donneront plusieurs implémentations de l'algorithme en question. Si les types de données qu'on connaît ne représentent pas d'une manière cohérente la réalité, on peut définir des types adaptés à notre contexte, on les appelle TAD.

Pour expliquer davantage ce que c'est qu'un Type Abstrait de Données, nous pouvons considérer par exemple le type « **Int** » dans le langage C. Pour utiliser des données de ce type, le langage C offre des conventions de manipulation couvrant l'écriture des constantes, la liste des opérations permises (+,-,*,...) ainsi que leurs propriétés. Il est donc tout à fait possible de manipuler des données de ce type sans savoir comment ils sont représentés en interne (nombre de bits, bit de signe...). C'est ça le concept de TAD qu'on utilise couramment sans le savoir [**Froidevaux et al., 1993**].

Avantages du concept

Les avantages de ce concept consistent en le fait qu'il facilite la conception, puisqu'on n'a pas à prendre en compte la programmation ; elle est faite une fois pour toutes quelle que soit la représentation choisie ultérieurement pour le type abstrait. Dans les chapitres suivants nous allons appliquer cette démarche.

2. Signature d'un TAD

Un Type Abstrait de Données est défini par une signature représentant sa syntaxe et les propriétés représentant la sémantique. La syntaxe d'un type abstrait de données est décrite par le concept de signature. La signature englobe les noms des opérations ainsi que les types des arguments mais ne définit pas les propriétés de celles-ci.

La signature d'un TAD est la donnée de :

* Noms de certains ensembles utilisés par le type (booléen, entier,...) : ces noms sont appelés sortes (c'est l'équivalent de types en programmation).

* Noms d'un certain nombre d'opérations et de leurs profils. Le profil précise l'ensemble de valeurs des arguments et des résultats.

Exemple de signature

Sortes *Vecteur*, *Element*, *Entier*

Opérations

Ième : Vecteur x Entier → Element

Change-ième : Vecteur x Entier x Element → Vecteur

Bornesup : Vecteur → Entier

Borneinf : Vecteur → Entier

Dans le premier exemple, nous utilisons les entiers pour numéroter les éléments du vecteur mais il serait peu pratique de donner la signature et les propriétés des entiers quand on définit les vecteurs. D'ailleurs la signature donnée précédemment est incomplète car rien n'est dit sur les calculs permis sur les entiers et les éléments. On se donne donc la possibilité de réutiliser des types déjà définis dans la définition d'un nouveau type. Donc l'exemple précédent aurait dû être écrit comme suit :

Sorte *Vecteur*

Utilise *Element*, *Entier*

Opérations

Ième : Vecteur x Entier → Element

Change-ième : Vecteur x Entier x Element → Vecteur

Bornesup : Vecteur → Entier

Borneinf : Vecteur → Entier

La signature du type vecteur est l'union des signatures des types utilisés, enrichie des nouveaux noms de sortes et d'opérations. Dans ce cas, on peut utiliser par exemple l'addition sur les entiers... On appelle **sorte définie** la ou les sortes correspondant aux nouveaux noms de sorte (tel que vecteur) et sorte prédéfinie la ou les sortes provenant des types utilisés.

Une opération est **interne** si elle rend un résultat d'une sorte définie (exp : change-ième). Toute valeur d'une sorte définie est le résultat d'une opération interne.

On dira qu'une opération est un **observateur** si elle a au moins un argument d'une sorte définie et si elle rend un résultat d'une sorte prédéfinie.

La signature définie ci-dessus n'est pas suffisante pour définir le TAD « Vecteur » car elle ne donne qu'une simple syntaxe. L'exemple ci-dessous montre plus l'insuffisance de la signature pour la définition du TAD.

Sorte R, S, T

Opérations

$O : R \times T \rightarrow S$

$P : R \times T \times S \rightarrow R$

$Q : R \rightarrow T$

$V : R \rightarrow T$

Pour compléter la définition du TAD, il est impératif d'ajouter la sémantique des opérations définies dans sa signature.

3. Description des propriétés d'un TAD

Les propriétés des opérations d'un TAD sont en général décrites sous forme d'axiomes.

Par exemple : $\text{BorneInf}(v) \leq i \leq \text{BorneSup}(v) \Rightarrow \text{Ième}(\text{changer_ième}(v, i, e), i) = e$

Où v, i, e sont des variables respectivement de sortes Vecteur, entier et element.

De plus, les opérations énoncées sont parfois des fonctions partielles non définies partout. Le domaine de définition d'une opération partielle est défini par une précondition.

La définition d'un type abstrait de données est donc composée d'une signature et d'un ensemble de propriétés (axiome + précondition).

Terminons l'exemple de sorte Vecteur qui a été proposé précédemment...

$\text{Vect} : \text{Entier} \times \text{Entier} \rightarrow \text{Vecteur}$ (retourne un vecteur où aucun élément n'est défini, les entiers c 'est les deux bornes)

$\text{Init} : \text{Vecteur} \times \text{Entier} \rightarrow \text{Booléen}$ (il faut aussi savoir si une certaine case du vecteur a été initialisée)

Donc la signature devient

Sorte Vecteur

Utilise *Element, Entier, booléen*

Opérations

$\text{Vect} : \text{Entier} \times \text{Entier} \rightarrow \text{Vecteur}$ (op interne) // donne un vecteur où aucun élément n'est défini.

$\text{Init} : \text{Vecteur} \times \text{Entier} \rightarrow \text{Booléen}$ (observateur) // permet de savoir si un élément a été associé à un certain indice.

$\text{Ième} : \text{Vecteur} \times \text{Entier} \rightarrow \text{Element}$ (observateur)

$\text{Change-ième} : \text{Vecteur} \times \text{Entier} \times \text{Element} \rightarrow \text{Vecteur}$ (op interne)

BorneSup : Vecteur $\rightarrow$ Entier (observateur)

BorneInf : Vecteur $\rightarrow$ Entier (observateur)

Préconditions

Axiomes

Init(vect(i,j),k)=faux

BorneInf(Vect(i,j))=i

BorneSup(Vect(i,j))=j

Init(v,i)=faux et borneInf(v) $\leq$ i $\leq$ borneSup(v) $\Rightarrow$ Init(change-ième(v,i,e),i)=vrai

BorneInf(v) $\leq$ i $\leq$ BorneSup(v) et i $\neq$ j $\Rightarrow$ Init(change-ième(v,i,e),j) = Init(v,j)

BorneInf(v) $\leq$ i $\leq$ BorneSup(v) et BorneInf(v) $\leq$ j $\leq$ BorneSup(v) et i $\neq$ j $\Rightarrow$ lème(Change-ième(v,i,e),j)=lème(v,j)

BorneInf(v) $\leq$ i $\leq$ BorneSup(v) $\Rightarrow$ lème(Change-ième(v,i,e),i)=e

BorneInf(Change-ième(v,i,e))=BorneInf(v)

BorneSup(Change-ième(v,i,e))=BorneSup(v) Avec v: Vecteur, i,j: entier, e: Element

La définition des propriétés des opérations implique une certaine rigueur et cohérence. Pour ceci deux propriétés doivent être respectées :

La Consistance : les axiomes ne doivent pas être contradictoires.

La Complétude : nombre suffisant d'axiomes pour décrire toutes les propriétés du type abstrait à spécifier.

CHAPITRE VII - LES LISTES

1. Définition

Les listes linéaires sont la forme la plus courante d'organisation des données. On les utilise pour organiser des données qui doivent être traitées de manière **séquentielle**. Elles doivent être évolutives, c'est à dire que l'on doit pouvoir **ajouter** et **supprimer** des éléments.

Une *liste* est une suite finie (éventuellement vide) d'éléments **de même type** repérés selon leur **rang** dans la liste [Froidevaux et al., 1993].

On remarque que l'ordre des éléments est fondamental. Mais attention, il ne s'agit pas d'un ordre sur les valeurs des éléments mais d'un ordre sur les places des éléments dans la liste. Deux visions des listes sont possibles :

Vision **itérative** qui considère une liste comme étant un ensemble d'éléments où chaque élément pointe vers l'élément suivant. Dans ce cas on se base sur les opérations d'accès et d'insertion.

Vision **réursive** qui définit une liste par son premier élément celui-ci pointe vers la liste des éléments suivants. Dans ce cas on se base sur les opérations Tete qui rend la première place de la liste et Fin qui rend la liste des éléments suivants. Pour éviter les redondances, on s'intéressera, dans ce chapitre, à plusieurs implémentations des « Listes itératives » et on traitera les « Listes récursives » qui sont équivalentes aux « Piles » dans le chapitre suivant.

2. Types abstraits « Liste »

a. Liste itérative

Signature

Sorte Liste, Place

Utilise : Entier (rang), Element (valeur)

Opérations :

Liste-vide : $\rightarrow$ Liste	<i>interne pour Liste</i>
Acces : Liste x Entier $\rightarrow$ Place	<i>observateur pour Liste et <u>interne pour Place</u></i>
Contenu : Place $\rightarrow$ Element	<i><u>observateur pour Place</u></i>
Longueur : Liste $\rightarrow$ Entier	<i>observateur pour Liste</i>
Supprimer : Liste x Entier $\rightarrow$ Liste	<i>interne pour Liste</i>
Insérer : Liste x Entier x Element $\rightarrow$ Liste	<i>interne pour Liste</i>
Succ : Place $\rightarrow$ Place	<i><u>interne pour Place</u></i>

Ième : Liste x Entier $\rightarrow$ Element*observateur pour Liste***Préconditions :****Axiomes :**

Longueur (Liste-vide)=0

Si $l \neq$ Liste-vide et $1 \leq k \leq \text{Longueur}(l)$ alors $\text{Longueur}(\text{supprimer}(l,k)) = \text{Longueur}(l) - 1$ Si $1 \leq k \leq \text{Longueur}(l) + 1$ alors $\text{Longueur}(\text{insérer}(l,k,e)) = \text{Longueur}(l) + 1$ $l \neq$ Liste-vide et $1 \leq k \leq \text{Longueur}(l)$ et $1 \leq i < k \Rightarrow \text{ième}(\text{Supprimer}(l,k),i) = \text{ième}(l,i)$ $l \neq$ Liste-vide et $1 \leq k < \text{Longueur}(l)$ et $k \leq i \leq \text{Longueur}(l) - 1 \Rightarrow$ $\text{ième}(\text{Supprimer}(l,k),i) = \text{ième}(l,i+1)$ $1 \leq k \leq \text{Longueur}(l) + 1$ et $1 \leq i < k \Rightarrow \text{ième}(\text{insérer}(l,k,e),i) = \text{ième}(l,i)$ $1 \leq k \leq \text{Longueur}(l) + 1$ et $i = k \Rightarrow \text{ième}(\text{insérer}(l,k,e),i) = e$ $1 \leq k \leq \text{Longueur}(l) + 1$ et $k < i \leq \text{Longueur}(l) + 1 \Rightarrow \text{ième}(\text{insérer}(l,k,e),i) = \text{ième}(l,i-1)$ **Pour Place** $l \neq$ liste-vide et $1 \leq k < \text{Longueur}(l) \Rightarrow \text{Succ}(\text{accès}(l,k)) = \text{accès}(l,k+1)$ *Avec l : Liste ; i,k : Entier et e : Element*

3. Représentation des listes

a. Représentation contigüe

Dans ce cas la liste est représentée par un tableau dont la i ème case est la i ème place. La longueur de la liste ne doit pas dépasser la longueur du tableau qui est surdimensionné. Donc pour manipuler la liste nous avons besoin de connaître sa longueur parce que les cases du tableau ne sont pas toutes significatives.

Liste itérative et représentation contigue

Développer les procédures insérer et supprimer.

Avantages

Inconvénients

Conclusion

b. Représentation chaînée

Dans cette représentation les éléments de la liste sont chaînés entre eux à l'aide des pointeurs. La liste est déterminée par l'adresse de son 1^{er} élément. La liste est représentée par un article contenant 2 champs <valeur, ptr>.

Remarques :

La liste vide est représentée par le pointeur Nil.

Si $l \neq \text{listevide}$ alors l contient l'@ de la tête de la liste.

Listes itératives et représentation chaînée

Avantages

Taille non limitée

Allocation dynamique (allocation et libération selon les besoins)

Facilité de la suppression et de l'insertion n'importe où dans la liste

Inconvénients

Accès séquentiel

Nécessite plus d'espace mémoire pour le chaînage (adresse du suivant)

Conclusion

Cette représentation est adaptée aux listes évolutives et/ou aux cas où on ne connaît pas le nombre d'éléments de la liste.

4. Autres types de listes**a. Liste circulaire**

Dans une liste circulaire on remplace, dans la dernière place de la liste, le pointeur à Nil par un pointeur vers la tête de la liste. Ce type de listes est utile pour représenter les files (que nous verrons dans les chapitres suivants).

Si l'on choisit la dernière place de la liste comme point d'entrée ; il suffit de parcourir un seul lien pour retrouver la tête de la liste.

b. Liste doublement chaînée

Dans la représentation classique, le parcours des listes est orienté dans un seul sens : du premier élément vers le dernier. Mais de nombreuses applications nécessitent de parcourir la liste à la fois vers l'avant et vers l'arrière, et dans ce cas on peut faciliter le traitement en rajoutant des pointeurs arrière ou chaînage arrière. On obtient une liste doublement chaînée. Chaque place de cette liste contient un pointeur vers la place suivante (successeur) et un pointeur vers la place précédente.

c. Exercice

On considère la liste doublement chaînée représentée comme suit :

LD=struct

Noeud=struct

First : ^Noeud

Val: Element

Last : ^Noeud

Pred: ^Noeud

Finstruct

Suiv: ^Noeud

Finstruct

1. Donner la procédure d'insertion en tête d'un élément dans une liste doublement chaînée L .
2. Donner la fonction qui permet de supprimer le dernier élément d'une liste doublement chaînée L et de retourner sa valeur. On suppose que la liste n'est pas vide.

